Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Generative AI for 3D Modeling

TGS Ref No: TGS-2023037544

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 8.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
8.0	15 August 2026	Rebuilt the course around Blender and an AI-assisted 3D modelling workflow; expanded concepts, activities, visuals and assessments.	Dr Alfred Ang

TABLE OF CONTENTS

DOCUMENT VERSION CONTROL RECORD....3
TABLE OF CONTENTS....4
How to Use This Guide....5
Course Overview....5
Learning Outcomes....5
TSC Knowledge and Abilities....5
Topic 1: Generative AI for 3D Design Concepts and Modeling....5
1.1 AI-assisted 3D workflow....5
1.2 From requirement to model brief....6
1.3 Prompt architecture for geometry....6
1.4 Reference-image literacy....7
1.5 Blender scene and data model....7
1.6 Coordinate systems, units and scale....7
1.7 Mesh anatomy and selection....8
1.8 Primitive-based blockout....8
1.9 Non-destructive modifiers....9
1.10 Blender Python and bpy....9
1.11 Error-feedback iteration....9
1.12 Design variation and selection....10
1.13 Prompt and asset provenance....10
1.14 Human review and responsible use....10
1.15 Evidence-driven activity practice....11
Activity 1: Translate a Design Requirement into an AI-Ready 3D Brief....12
Activity 2: Generate a Blender Gold-Sphere Scene with AI-Assisted Python....13
Activity 3: Iterate and Repair an AI-Generated Blender Script....15
Activity 4: Develop and Select Three AI-Assisted Product Variants....16
Topic 2: AI-Assisted 3D Model Analysis, Validation and Refinement....16
2.1 Validation framework....16
2.2 Silhouette and proportion review....17
2.3 Topology quality....17
2.4 Manifold and watertight geometry....18
2.5 Normals and face orientation....18
2.6 Transform hygiene....18
2.7 Cleanup strategy....19
2.8 Retopology approaches....19
2.9 Polygon budgets and level of detail....20
2.10 UV coordinates and seams....20
2.11 PBR materials....20
2.12 AI-generated textures....21

2.13 Lighting as a validation tool....21
2.14 Change control and review....22
2.15 Quality dashboard....22
Activity 5: Inspect an AI-Generated Mesh and Build a Validation Baseline....23
Activity 6: Repair Non-Manifold Geometry, Normals and Duplicate Data....24
Activity 7: Retopologize and Optimize an AI Mesh for Its Target Use....25
Activity 8: UV-Unwrap and Apply AI-Assisted PBR Materials....26
Topic 3: 3D Model Optimization, Standards and Production Workflows....26
3.1 Target-use matrix....26
3.2 Technical conventions for 3D assets....27
3.3 Dimension and tolerance evidence....27
3.4 Render engines and delivery intent....28
3.5 Camera and composition....28
3.6 Colour management....28
3.7 Export formats....29
3.8 Game-ready optimization....29
3.9 3D-print preparation....30
3.10 Animation-ready organization....30
3.11 Documentation package....30
3.12 Automated checks....31
3.13 Stakeholder design review....31
3.14 Organisational alignment....32
3.15 Final acceptance and release....32
Activity 9: Create a Neutral Studio Render and Visual QA Set....33
Activity 10: Prepare and Verify a Game/Visualization Export....34
Activity 11: Prepare and Verify a 3D-Printable STL....35
Activity 12: Capstone: Deliver an AI-Assisted 3D Asset Review Package....36
Quick Reference....37
Assessment and Support....37

How to Use This Guide

Use this guide for the detailed click-by-click procedures. The slide deck is concept-led and intentionally does not duplicate these procedures. Work on copies of the supplied Blender assets, save incremental versions, and capture the evidence named in each activity.

Course Overview

This four-day WSQ course combines generative AI with Blender to create, inspect, refine, validate and deliver accountable 3D assets for visualisation, games, animation and 3D printing.

Learning Outcomes

- LO1: Devise metrics for AI-assisted 3D design outputs and formulate solutions to modelling problems.
- LO2: Analyse Blender models for accuracy, topology quality and alignment with design requirements.
- LO3: Review and prepare 3D assets for organisational needs, applicable standards and production delivery.

TSC Knowledge and Abilities

- K1: Symbols, standards and conventions used in technical drawings and specifications across fields and regions.
- K2: Methods of technical drawing using instruments and digital systems.
- K3: Types of technical drawings and specifications.
- K4: Technical drawing standards.
- K5: Principles of design documentation.
- K6: Information technology concepts that facilitate technical drawing software use.
- A1: Devise metrics and guidelines for the production of technical drawings.
- A2: Formulate solutions for problems encountered while developing drawings and specifications.
- A3: Review technical drawings for accuracy and adherence to specifications.
- A4: Analyse drawings and specifications to determine construction needs and deliver design solutions.
- A5: Review drawings and changes for relevance to organisational strategies.
- A6: Review drawings and specifications for conformance to relevant standards.

Topic 1: Generative AI for 3D Design Concepts and Modeling

Briefs · prompts · Blender foundations · script-assisted construction · design variants

1.1 AI-assisted 3D workflow

Generative AI accelerates ideation and repetitive construction; Blender remains the environment for inspection, correction and accountable delivery.

- **Brief:** Define purpose, audience, dimensions and constraints.
- **Generate:** Produce concepts, scripts or reference images.
- **Refine:** Edit geometry, topology and materials in Blender.
- **Validate:** Measure, inspect and document before release.

Applied example:

A product designer asks AI for a Blender blockout script, runs it in a duplicate file, then checks scale, silhouette and topology before continuing.

Review decisions:

- Choose the AI contribution before choosing a tool.
- Keep a human approval gate after every generated output.
- Preserve prompt, version and source evidence.
- Treat generated geometry as unverified input.

1.2 From requirement to model brief

A precise model brief converts ambiguous intent into testable geometry, appearance and delivery requirements.

- **Purpose:** Game prop, print, visualisation, animation or marketing.
- **Geometry:** Dimensions, proportions, silhouette and required moving parts.
- **Appearance:** Material family, colour, finish and lighting context.
- **Delivery:** File format, polygon budget, scale and evidence.

Applied example:

Instead of 'make a modern speaker', specify a 180 mm desktop speaker, rounded enclosure, front grille, separate control knob, neutral materials and glTF delivery.

Review decisions:

- Separate must-have constraints from creative preferences.
- State units and coordinate assumptions explicitly.
- Define acceptance evidence before generation.
- Identify what must remain editable.

1.3 Prompt architecture for geometry

Effective prompts describe topology-relevant structure, not only visual style.

- **Object:** Name the object and its functional parts.
- **Form:** Specify primitives, proportions and symmetry.
- **Topology:** State manifold, quad flow or low-poly constraints.
- **Output:** Request a Blender Python script or a reference image deliberately.

Applied example:

A useful request names a body, bezel, knob and feet; states dimensions; requests bevel and weighted-normal modifiers; and asks the script to save no files automatically.

Review decisions:

- Use Blender terms when asking for Blender actions.
- Break complex objects into reusable components.
- Request named objects and collections.
- Ask for safe, rerunnable scripts.

1.4 Reference-image literacy

A reference image is evidence of appearance, not proof of hidden geometry or real-world dimensions.

- **View:** Identify perspective, focal length and occluded surfaces.
- **Scale:** Find a known dimension or reference object.
- **Symmetry:** Decide what can be mirrored.
- **Uncertainty:** Mark inferred surfaces and assumptions.

Applied example:

A single three-quarter product image supports silhouette and material decisions, but the underside, wall thickness and exact depth remain assumptions.

Review decisions:

- Prefer orthographic front/side/top references when available.
- Do not trace perspective distortion as real geometry.
- Document assumptions for unseen surfaces.
- Use multiple images to triangulate form.

1.5 Blender scene and data model

Blender separates objects, object data, materials, collections and scene settings; understanding that structure prevents destructive edits.

- **Object:** Transform container positioned in the scene.
- **Mesh data:** Vertices, edges and faces shared by an object.
- **Collection:** Logical grouping for visibility and export.
- **Scene:** World, camera, render and unit settings.

Applied example:

AI_Device_BODY and AI_Device_KNOB live in a WORKING collection; a duplicate approved collection is locked before risky cleanup.

Review decisions:

- Name objects by purpose, not primitive type.
- Use collections for source, working and approved states.
- Apply transforms only when the downstream operation requires it.
- Save incremental versions.

1.6 Coordinate systems, units and scale

A model can look correct in the viewport and still fail in manufacturing or another application because units and axes are wrong.

- **Units:** Metric or imperial with an explicit scale.
- **Axes:** Blender Z-up; target engines may use Y-up.
- **Origin:** Defines placement, rotation and export behaviour.
- **Transforms:** Location, rotation and scale must be deliberate.

Applied example:

A 180 mm prototype is modeled as 0.18 m, origin at its base centre, scale 1/1/1, and export conversion documented.

Review decisions:

- Set scene units before dimensional work.
- Place origins at meaningful assembly or pivot points.
- Apply scale before bevel, physics and export when appropriate.
- Test import in the target system.

1.7 Mesh anatomy and selection

Vertices, edges, faces, loops and islands are the editable structure that turns an AI concept into controllable geometry.

- **Vertices:** Points that define positions.
- **Edges:** Connections controlling flow and boundaries.
- **Faces:** Surface polygons with winding and normals.
- **Loops:** Continuous selections used for form and topology control.

Applied example:

A supporting loop around a product opening controls its edge sharpness under subdivision without adding an unrelated separate object.

Review decisions:

- Choose object mode for whole-object transforms.
- Use edit mode for component-level changes.
- Select by topology, not only by screen rectangle.
- Inspect hidden and internal geometry.

1.8 Primitive-based blockout

Simple primitives establish proportion and hierarchy faster and more safely than prematurely detailed meshes.

- **Box:** Hard-surface bodies and enclosures.
- **Cylinder:** Knobs, ports and radial features.
- **Plane:** Profiles, panels and reference surfaces.
- **Curve:** Cables, trims and controlled sweeps.

Applied example:

A desktop device begins as a beveled box, a cylinder knob and a recessed panel; the team approves silhouette before adding vents.

Review decisions:

- Validate silhouette from multiple views.
- Keep components separate while proportions change.
- Use symmetry before manual duplication.
- Delay bevel and micro-detail.

1.9 Non-destructive modifiers

A modifier stack records intent and preserves editability while transforming the visible result.

- **Mirror:** Symmetric construction from one side.
- **Bevel:** Manufacturable edge treatment and highlights.
- **Subdivision:** Smooth form from a control cage.
- **Solidify:** Thickness from a surface.

Applied example:

Mirror → Bevel → Weighted Normal produces a symmetric hard-surface body with controlled highlights and editable dimensions.

Review decisions:

- Order modifiers by dependency.
- Use conservative viewport levels for performance.
- Name critical modifiers.
- Apply only at a controlled handoff point.

1.10 Blender Python and bpy

Blender's Python API can create and edit data deterministically, but generated scripts must be scoped, readable and reversible.

- **Data API:** Direct access to objects, meshes, materials and scenes.
- **Operators:** Context-sensitive actions similar to UI commands.
- **Idempotence:** A safe rerun reaches the same intended state.
- **Logging:** Names and messages make failures diagnosable.

Applied example:

A script removes only objects in collection AI_GENERATED, rebuilds the blockout, names every part and prints a completion summary.

Review decisions:

- Ask the model to target the installed Blender version.
- Delete or reuse only named generated objects.
- Avoid automatic overwrite or save calls.
- Run in a copy and inspect the console.

1.11 Error-feedback iteration

An error message is structured evidence: capture the traceback, locate the failing line, form a hypothesis and test one repair at a time.

- **Capture:** Copy the full traceback and Blender version.
- **Classify:** Syntax, missing data, context or API mismatch.
- **Repair:** Change the smallest relevant section.
- **Retest:** Run in a clean duplicate and compare results.

Applied example:

A context error from bpy.ops is repaired by switching to direct data creation rather than repeatedly changing selection state.

Review decisions:

- Do not ask AI to rewrite everything after one error.
- Provide the exact error and intended result.
- Inspect the proposed fix before running.
- Record successful changes.

1.12 Design variation and selection

Generative breadth is valuable only when variants are compared against explicit requirements.

- **Diverge:** Generate meaningful alternatives.
- **Normalize:** Use the same view, scale and lighting.
- **Score:** Apply weighted criteria.
- **Converge:** Select and record the decision.

Applied example:

Three enclosures are scored on usability, manufacturability, brand fit and production cost; the winner is not simply the most visually novel.

Review decisions:

- Vary one major design assumption per variant.
- Use common camera and materials for fair comparison.
- Weight functional criteria above novelty.
- Preserve rejected variants with reasons.

1.13 Prompt and asset provenance

A production asset needs traceability across input references, prompts, generated outputs, manual edits and approvals.

- **Source:** Reference owner, licence and retrieval date.
- **Prompt:** Exact request and model/tool context.
- **Changes:** Manual fixes and derived versions.
- **Approval:** Reviewer, date and accepted purpose.

Applied example:

The asset register links the prompt, concept image, Blender file v03, validation checklist and final export.

Review decisions:

- Store prompts beside the activity evidence.
- Mark generated reference images as concept visuals.
- Separate reusable source from final derivative work.
- Never assume AI output is rights-cleared.

1.14 Human review and responsible use

AI output must be reviewed for safety, bias, plausibility, intellectual property and fitness for the stated use.

- **Accuracy:** Does the model match dimensions and function?
- **Safety:** Could geometry create hazards or misleading claims?
- **Rights:** Are references and outputs legally usable?
- **Disclosure:** Would a stakeholder mistake concept imagery for verified evidence?

Applied example:

A generated medical-device concept may support ideation but cannot be represented as engineered, tested or clinically approved.

Review decisions:

- Label concept visuals clearly.
- Escalate safety-critical and regulated uses.
- Remove sensitive information from prompts.
- Require human sign-off before release.

1.15 Evidence-driven activity practice

A strong activity submission proves process, decision quality and acceptance—not just a final attractive screenshot.

- **Brief:** What was requested.
- **Process:** Prompt, settings and revisions.
- **Evidence:** Screenshots and measurements.
- **Acceptance:** Checklist and reflection.

Applied example:

The sample presentation shows the brief, AI contribution, Blender validation, final render and a signed acceptance checklist.

Review decisions:

- Capture before/after from the same view.
- Include Blender file and exported output.
- Name evidence consistently.
- Explain one issue and its fix.

Activity 1: Translate a Design Requirement into an AI-Ready 3D Brief

Goal: Create measurable geometry, appearance and delivery criteria before using AI.

Scenario: A retail innovation team needs a compact desktop product visual for stakeholder review and later prototyping.

Output: A one-page model brief, prompt specification and acceptance scorecard.

Step-by-step

1. Open the activity brief and identify the intended audience, use case and final delivery format.
2. Write the product purpose in one sentence without naming a software tool.
3. List the primary parts that must remain separately editable in Blender.
4. Set the target dimensions in millimetres and identify one critical clearance.
5. Describe the silhouette using primitive geometry terms such as box, cylinder, curve and symmetry.
6. Define the material family, colour palette and finish without relying only on subjective adjectives.
7. Choose three required evidence views: front, side and three-quarter perspective.
8. Set delivery requirements for the Blender source, presentation render and one exchange format.
9. Create a four-criterion scorecard covering form, structure, appearance and delivery.
10. Review the brief for ambiguous words and replace each with a measurable statement.
11. Save the completed brief and record its version in the evidence presentation.

Acceptance checklist

- Purpose and audience are explicit
- Dimensions and units are stated
- Editable parts are named
- Acceptance criteria are measurable
- Delivery formats are defined

Evidence to submit

- Completed model brief
- Prompt specification
- Acceptance scorecard
- One-slide decision summary

Activity folder: activities/01-…/

Activity 2: Generate a Blender Gold-Sphere Scene with AI-Assisted Python

Goal: Use the video workflow to request, inspect, run and verify a small Blender Python script.

Scenario: The team wants a repeatable proof-of-concept showing how natural-language intent can create named geometry and a material in Blender.

Output: A safe Blender script, a generated gold sphere scene and an execution evidence presentation.

Step-by-step

1. Open the starter Blender file and save a working copy inside the activity folder.
2. Confirm the installed Blender version from Help > About Blender and record it.
3. Ask the AI assistant to write a Blender Python script for that version which creates a UV sphere named AI_GoldSphere, gives it a gold Principled BSDF material, adds a floor, light and camera, and does not save or overwrite files.
4. Review the script for broad delete commands, file-writing commands and unnamed objects before running it.
5. Switch to the Scripting workspace and create a new text block.
6. Paste the reviewed script into the text editor.
7. Run the script and watch the status bar and system console for errors.
8. Return to the Layout workspace and switch to Material Preview or Rendered view.
9. Verify that AI_GoldSphere exists, uses the gold material and is positioned above the floor.
10. Check the Outliner for clear names and confirm the sphere scale is 1, 1, 1.
11. Render one camera view and save the image in the evidence folder.
12. Save the Blender file as Activity-02-Gold-Sphere-Completed.blend.
13. Update the sample presentation with the prompt, script review, result and verification evidence.

Reference Blender Python script

```
import bpy

COLLECTION = 'AI_GENERATED'
# Remove only objects previously generated by this activity.
old = bpy.data.collections.get(COLLECTION)
if old:
    for obj in list(old.objects):
        bpy.data.objects.remove(obj, do_unlink=True)
else:
    old = bpy.data.collections.new(COLLECTION)
    bpy.context.scene.collection.children.link(old)

def move_to_collection(obj):
    for col in list(obj.users_collection):
        col.objects.unlink(obj)
    old.objects.link(obj)

bpy.ops.mesh.primitive_uv_sphere_add(segments=64, ring_count=32, location=(0, 0, 1.1))
sphere = bpy.context.active_object
sphere.name = 'AI_GoldSphere'
move_to_collection(sphere)
bpy.ops.object.shade_smooth()
mat = bpy.data.materials.get('MAT_Gold') or bpy.data.materials.new('MAT_Gold')
mat.use_nodes = True
bsdf = mat.node_tree.nodes.get('Principled BSDF')
bsdf.inputs['Base Color'].default_value = (0.83, 0.49, 0.08, 1.0)
bsdf.inputs['Metallic'].default_value = 1.0
bsdf.inputs['Roughness'].default_value = 0.22
sphere.data.materials.clear()
sphere.data.materials.append(mat)
print('Created AI_GoldSphere with MAT_Gold; file not saved automatically.')
```

Acceptance checklist

- Script targets the installed Blender version
- No automatic file overwrite
- Named sphere and material exist
- Scale and placement verified
- Render and Blender source saved

Evidence to submit

- Exact prompt
- Reviewed script
- Outliner screenshot
- Rendered result
- Completed .blend file

Activity folder: activities/02-…/

Activity 3: Iterate and Repair an AI-Generated Blender Script

Goal: Diagnose a controlled script failure and use evidence-driven AI feedback to repair it safely.

Scenario: A generated script fails after a Blender update and leaves partial objects in the scene.

Output: A repaired rerunnable script, an error log and a before/after diagnosis.

Step-by-step

1. Open the starter file and save a diagnostic working copy.
2. Open the supplied faulty script and read its intended outcome before running it.
3. Run the script once and copy the complete traceback, including the failing line.
4. Classify the failure as syntax, missing data, context/mode or API-version mismatch.
5. Inspect the Outliner for any partial objects created before the failure.
6. Ask the AI assistant to repair only the failing section, providing the Blender version, full traceback and intended outcome.
7. Review the proposed patch for new destructive operations or unrelated rewrites.
8. Apply the smallest change and rerun in a clean duplicate of the starter file.
9. Run the repaired script a second time to test idempotence and check for duplicates.
10. Compare object names, modifier stacks and geometry against the expected example.
11. Record the root cause, repair and regression checks in the presentation.
12. Save the final script and Blender file with a new version number.

Acceptance checklist

- Full traceback captured
- Root cause classified
- Patch is narrowly scoped
- Second run creates no duplicates
- Expected geometry and modifiers verified

Evidence to submit

- Error screenshot
- Root-cause statement
- Before/after code excerpt
- Two-run Outliner check
- Final script and .blend

Activity folder: activities/03-…/

Activity 4: Develop and Select Three AI-Assisted Product Variants

Goal: Generate meaningful design alternatives and select one using weighted acceptance criteria.

Scenario: Stakeholders want options for the desktop product but need a defendable recommendation, not a gallery of arbitrary shapes.

Output: Three normalized Blender blockouts, a weighted decision matrix and a selected direction.

Step-by-step

1. Duplicate the approved brief from Activity 1 and lock the must-have constraints.
2. Choose one major design assumption to vary: stance, control placement or enclosure profile.
3. Create or generate three blockout variants while keeping scale and required parts constant.
4. Place each variant in its own named collection: VAR_A, VAR_B and VAR_C.
5. Set the same material, camera, focal length and lighting for all three variants.
6. Capture front, side and three-quarter views for each variant.
7. Weight the criteria for functional fit, manufacturability, visual alignment and editability.
8. Score each variant and cite the evidence behind every score.
9. Run a sensitivity check by changing the highest weight by ten percentage points.
10. Select the preferred variant and record one trade-off and one mitigation.
11. Archive rejected variants without deleting them.
12. Update the evidence presentation with comparable views and the decision matrix.

Acceptance checklist

- Three variants differ meaningfully
- Common camera and lighting used
- Criteria total 100%
- Scores cite evidence
- Recommendation includes trade-off

Evidence to submit

- Nine comparable views
- Weighted matrix
- Sensitivity check
- Recommendation and mitigation
- Variant collections

Activity folder: activities/04-…/

Topic 2: AI-Assisted 3D Model Analysis, Validation and Refinement

Inspection · topology · normals · retopology · UVs · materials · controlled repair

2.1 Validation framework

Validation checks form, structure, dimensions, appearance and delivery requirements in a repeatable order.

- **Form:** Silhouette and proportions.
- **Structure:** Topology, normals and manifold status.
- **Appearance:** UVs, materials and shading.

- **Delivery:** Naming, scale, format and evidence.

Applied example:

A visually appealing model fails because its wall is open and its scale is 1000× too large; validation catches both before export.

Review decisions:

- Validate from broad shape to small defects.
- Measure against the brief, not memory.
- Separate blocking defects from refinements.
- Record pass/fail evidence.

2.2 Silhouette and proportion review

The silhouette communicates primary form; orthographic views expose proportion errors that perspective views can hide.

- **Front:** Width, symmetry and feature alignment.
- **Side:** Depth, taper and profile.
- **Top:** Footprint and spacing.
- **Perspective:** Overall visual coherence.

Applied example:

A handle reads well in perspective but blocks the opening in side view; the review prioritises functional clearance.

Review decisions:

- Compare against calibrated references.
- Use overlays or guides for key dimensions.
- Check negative space as carefully as solids.
- Review at target viewing distance.

2.3 Topology quality

Good topology supports the intended deformation, shading, editing and export while using no more geometry than necessary.

- **Edge flow:** Loops follow form and deformation.
- **Density:** Resolution is allocated where curvature or silhouette needs it.
- **Poles:** Higher-valence vertices are placed away from critical areas.
- **Faces:** Quads, triangles and n-gons are used intentionally.

Applied example:

A product body uses clean supporting loops at openings and low density on flat panels; a decorative triangulated insert remains separate.

Review decisions:

- Topology is purpose-dependent.

- Static hard-surface assets tolerate different flow from characters.
- Inspect subdivision behaviour.
- Avoid hidden internal geometry.

2.4 Manifold and watertight geometry

A manifold mesh forms an unambiguous closed surface; this is critical for printing, booleans and reliable downstream processing.

- **Boundary:** Open edges indicate holes.
- **Non-manifold:** Edges shared by the wrong number of faces.
- **Internal:** Faces trapped inside a volume.
- **Thickness:** Walls must meet the target process minimum.

Applied example:

A fish-shaped container has an unintended seam along the belly; merging boundary vertices and rebuilding faces closes the volume.

Review decisions:

- Use Select Non-Manifold as a diagnostic, not an automatic cure.
- Repair small gaps before remeshing.
- Check disconnected shells.
- Recalculate volume after repair.

2.5 Normals and face orientation

Normals define the direction surfaces face and influence shading, culling and export.

- **Face normals:** Outward direction of polygon faces.
- **Vertex normals:** Interpolated direction used for smooth shading.
- **Auto Smooth:** Controls splitting at sharp angles.
- **Weighted normals:** Biases shading toward larger faces.

Applied example:

Red face-orientation patches reveal inverted faces near a boolean cut; correction removes black shading after export.

Review decisions:

- Enable Face Orientation to spot flipped faces.
- Recalculate outside only after topology is valid.
- Mark sharp edges deliberately.
- Test in the target renderer.

2.6 Transform hygiene

Applied transforms make geometric operations predictable, but application must respect pivots, animation and linked data.

- **Location:** Placement relative to origin.
- **Rotation:** Orientation relative to axes.

- **Scale:** Object multiplier over mesh coordinates.
- **Origin:** Pivot and placement reference.

Applied example:

A bevel width varies across two objects because one has unapplied non-uniform scale; applying scale restores consistent treatment.

Review decisions:

- Check dimensions before applying scale.
- Keep origin intentional.
- Avoid negative scale for final delivery.
- Document exceptions.

2.7 Cleanup strategy

Repair the smallest root cause that restores requirements; do not destroy useful structure with indiscriminate remeshing.

- **Diagnose:** Identify cause and affected region.
- **Protect:** Duplicate or version the source.
- **Repair:** Merge, dissolve, rebuild or remesh deliberately.
- **Verify:** Repeat the original failing test.

Applied example:

A dense AI mesh needs only a local opening repair and decimation of hidden detail—not a complete voxel remesh.

Review decisions:

- Start with duplicate vertices and internal faces.
- Preserve silhouette while simplifying.
- Use local repair before global remesh.
- Compare statistics before and after.

2.8 Retopology approaches

Retopology rebuilds surface structure for the target use; the right method balances speed, control and preservation.

- **Manual:** Highest control for key loops and deformation.
- **Shrinkwrap:** Projects a clean cage onto a source.
- **Remesh:** Fast uniform topology with less semantic control.
- **Decimate:** Reduces faces while retaining triangulated shape.

Applied example:

A static visualisation asset may use controlled decimation; an animated creature needs manual quad flow around joints.

Review decisions:

- Define target use before selecting a method.
- Protect high-detail source.
- Place loops around deformation and openings.
- Bake detail when reducing density.

2.9 Polygon budgets and level of detail

A polygon budget is a performance and delivery constraint tied to screen size, platform and use—not a universal number.

- **Silhouette:** Spend polygons where the outline changes.
- **Deformation:** Support joints and bending.
- **Materials:** Avoid geometry for detail that textures can carry.
- **LOD:** Prepare lower-detail variants for distance.

Applied example:

A hero product render keeps fine geometry; a real-time background prop uses baked normals and a lower triangle count.

Review decisions:

- Measure triangles after triangulation.
- Benchmark in the target environment.
- Use consistent LOD naming.
- Avoid invisible internal detail.

2.10 UV coordinates and seams

UVs map a 3D surface to 2D texture space; seams determine where the surface is cut and unfolded.

- **Seams:** Cuts placed along hidden or logical boundaries.
- **Islands:** Contiguous UV regions.
- **Distortion:** Stretching between model and map.
- **Packing:** Efficient use of texture space.

Applied example:

A product body hides its main seam on the underside and allocates more UV area to the front control panel.

Review decisions:

- Apply scale before unwrapping.
- Place seams where discontinuity is acceptable.
- Equalise texel density for comparable surfaces.
- Leave padding for mipmaps and baking.

2.11 PBR materials

Physically based materials describe how light interacts with a surface through a small set of meaningful parameters.

- **Base colour:** Diffuse/reflective colour without lighting baked in.
- **Roughness:** Micro-surface spread of reflections.

- **Metallic:** Dielectric versus conductor behaviour.
- **Normal:** Fine directional detail without new silhouette geometry.

Applied example:

A powder-coated enclosure uses metallic 0, medium-high roughness and subtle normal detail; a metal knob uses metallic 1 with lower roughness.

Review decisions:

- Use plausible parameter ranges.
- Do not paint shadows into base colour.
- Keep colour data and non-colour data settings correct.
- Test under neutral lighting.

2.12 AI-generated textures

AI can accelerate texture ideation, but images must be made tileable, channel-correct, rights-aware and free of baked perspective.

- **Prompt:** Describe material, scale and surface condition.
- **Tile:** Remove seams and edge discontinuity.
- **Derive:** Create roughness/normal carefully.
- **Validate:** Inspect repeats, scale and physical plausibility.

Applied example:

A generated brushed-plastic reference is cleaned into a seamless base colour, then roughness is authored from measured visual cues.

Review decisions:

- Request flat, evenly lit material references.
- Remove recognisable logos or text.
- Use separate masks for controlled variation.
- Preserve provenance.

2.13 Lighting as a validation tool

Neutral lighting reveals material and surface quality; dramatic lighting should come after defects are understood.

- **Key:** Defines form and main highlight.
- **Fill:** Controls contrast and shadow readability.
- **Rim:** Separates silhouette from background.
- **Environment:** Provides consistent reflections.

Applied example:

A grazing key light exposes pinching around a boolean that was invisible under flat viewport shading.

Review decisions:

- Validate under a neutral studio setup.

- Rotate light to reveal shading artefacts.
- Use consistent exposure across variants.
- Separate evidence renders from marketing renders.

2.14 Change control and review

Every major repair should be reviewable against the previous version and the design brief.

- **Baseline:** Freeze the incoming AI model.
- **Change:** Name the repair and reason.
- **Compare:** Use consistent views and metrics.
- **Approve:** Record reviewer and acceptance.

Applied example:

v01 raw → v02 manifold repair → v03 retopology → v04 UV/material; each version records changed statistics.

Review decisions:

- Use incremental filenames.
- Avoid overwriting the raw source.
- Keep a change log.
- Recheck downstream effects.

2.15 Quality dashboard

A concise dashboard turns technical checks into an approval decision stakeholders can understand.

- **Geometry:** Dimensions, manifold, normals and polygon count.
- **Appearance:** UV stretch, materials and render consistency.
- **Delivery:** Naming, format and round-trip test.
- **Risk:** Open issues, owner and disposition.

Applied example:

The dashboard marks 11/12 checks passed and leaves 'minimum wall thickness' amber pending a print-process decision.

Review decisions:

- Use thresholds tied to the brief.
- Show current value and status.
- Link evidence to each result.
- Never hide a failed check in an average score.

Activity 5: Inspect an AI-Generated Mesh and Build a Validation Baseline

Goal: Measure geometry, transforms, topology and evidence before making repairs.

Scenario: An external AI mesh arrives with no quality record and must be assessed before refinement.

Output: A frozen baseline Blender file, defect register and validation dashboard.

Step-by-step

1. Copy the starter asset to a BASELINE folder and mark it read-only or preserve it unchanged.
2. Open a working duplicate and record Blender version, object count and collection names.
3. Check scene units, object dimensions, location, rotation, scale and origin.
4. Enable viewport statistics and record vertices, edges, faces and triangles.
5. Inspect the model in front, side, top and perspective views.
6. Use Face Orientation to check outward-facing surfaces.
7. In Edit Mode, use Select > Select All by Trait > Non-Manifold and count affected elements.
8. Check for disconnected shells, internal faces and duplicate vertices.
9. Review modifier stacks and note any warnings or unapplied dependencies.
10. Capture consistent screenshots of every blocking defect.
11. Classify findings as blocker, major, minor or observation.
12. Populate the validation dashboard without repairing the mesh yet.
13. Save the working inspection file separately from the baseline.

Acceptance checklist

- Baseline preserved
- Units and transforms recorded
- Topology statistics captured
- Non-manifold and normals checks completed
- Findings have severity and evidence

Evidence to submit

- Baseline metadata
- Statistics screenshot
- Face Orientation view
- Non-manifold selection
- Defect register

Activity folder: activities/05-…/

Activity 6: Repair Non-Manifold Geometry, Normals and Duplicate Data

Goal: Apply local, reversible repairs and prove that the original failing checks now pass.

Scenario: The inspected model has a belly seam, flipped faces and doubled vertices that cause shading and export defects.

Output: A repaired manifold model with before/after evidence and regression results.

Step-by-step

1. Duplicate the working object and suffix the copy _REPAIR while preserving the baseline.
2. Apply scale only after confirming the dimensions and recording the reason.
3. Enter Edit Mode and run Merge by Distance with a conservative threshold.
4. Review the status message and record how many vertices were merged.
5. Use Non-Manifold selection to isolate the open boundary.
6. Rebuild the smallest necessary faces or bridge compatible edge loops.
7. Inspect for internal faces around the repaired area and delete only confirmed internal geometry.
8. Select all faces and recalculate normals outside.
9. Enable Face Orientation and confirm no unexpected inward-facing patches remain.
10. Apply smooth shading and configure sharp edges or normals deliberately.
11. Repeat the non-manifold, dimensions and silhouette checks from Activity 5.
12. Compare polygon statistics and note any material or UV impact.
13. Save a new version and update the defect register to closed or open.

Acceptance checklist

- Baseline remains unchanged
- Merge threshold documented
- Boundary repaired locally
- Normals face outward
- Regression checks pass

Evidence to submit

- Before/after Face Orientation
- Merged-vertex count
- Repaired boundary close-up
- Regression table
- Versioned Blender file

Activity folder: activities/06-…/

Activity 7: Retopologize and Optimize an AI Mesh for Its Target Use

Goal: Select a retopology approach, meet a polygon budget and preserve critical form.

Scenario: A dense generated mesh must become a responsive real-time asset without losing its recognizable silhouette.

Output: A target-use retopology, statistics comparison and feature-retention evidence.

Step-by-step

1. State the target platform, camera distance, animation need and triangle budget.
2. Duplicate the repaired source and preserve it as the high-detail reference.
3. Identify silhouette-critical edges, openings and deformation regions.
4. Choose manual retopology, Shrinkwrap, Remesh or Decimate and justify the choice.
5. Build or generate the lower-density surface while keeping it aligned to the source.
6. Concentrate geometry at curvature changes and reduce density on broad flat areas.
7. Inspect poles, long thin faces and edge flow around openings.
8. Triangulate a duplicate to preview the target-engine result.
9. Compare front, side and perspective silhouettes against the source.
10. Record vertex, face and triangle reduction percentages.
11. Run the manifold and normal checks again.
12. Document one retained feature, one simplified feature and one remaining limitation.
13. Save the optimized model as a separate version.

Acceptance checklist

- Target use and budget stated
- Method justified
- Silhouette retained
- Topology checks pass
- Reduction metrics recorded

Evidence to submit

- High/low overlay
- Wireframe comparison
- Statistics table
- Triangulated preview
- Limitations note

Activity folder: activities/07-…/

Activity 8: UV-Unwrap and Apply AI-Assisted PBR Materials

Goal: Create usable UVs and physically plausible materials from AI-assisted references.

Scenario: The optimized asset needs a consistent product finish and texture set suitable for review and export.

Output: A UV-mapped Blender asset, validated material channels and texture evidence.

Step-by-step

1. Apply scale after verifying the model dimensions.
2. Identify hidden or logical boundaries for UV seams.
3. Mark seams around openings, underside transitions and detachable parts.
4. Unwrap the model and inspect islands using a checker texture.
5. Correct stretched regions and equalise texel density across comparable surfaces.
6. Pack islands with appropriate padding and preserve intentional overlaps only.
7. Review the AI-assisted texture reference for baked lighting, text, logos and visible seams.
8. Clean the base-colour image into an evenly lit, tileable material where required.
9. Create a Principled BSDF material and connect base colour, roughness and normal channels using the correct colour-space settings.
10. Tune plausible metallic and roughness values under neutral lighting.
11. Inspect UV seams, repeated patterns and normal-map strength at close range.
12. Record texture provenance, resolution and any manual edits.
13. Save the packed/linked asset and update the evidence presentation.

Acceptance checklist

- Seams follow logical boundaries
- Checker shows acceptable distortion
- Texel density is consistent
- Colour spaces are correct
- Texture provenance recorded

Evidence to submit

- UV layout screenshot
- Checker test
- Shader graph
- Neutral-light material render
- Texture register

Activity folder: activities/08-…/

Topic 3: 3D Model Optimization, Standards and Production Workflows

Standards · rendering · export · interoperability · documentation · capstone delivery

3.1 Target-use matrix

The same source model needs different preparation for games, 3D printing, animation and visualisation.

- **Games:** Real-time triangle, texture and draw-call budgets.
- **Printing:** Watertight volume, wall thickness and physical scale.
- **Animation:** Deformation topology, pivots and naming.
- **Visualisation:** High-quality shading, camera and colour management.

Applied example:

The approved master produces glTF for web, STL for print validation and a high-resolution Blender render file.

Review decisions:

- Choose the target before optimisation.
- Create delivery variants from a controlled master.
- Do not assume one export satisfies every use.
- Test in the destination.

3.2 Technical conventions for 3D assets

Consistent naming, units, axes, origins, hierarchy and metadata act as the conventions that make digital models understandable across teams.

- **Naming:** Role-based object and material names.
- **Units:** Declared measurement system and scale.
- **Axes:** Orientation and forward/up convention.
- **Hierarchy:** Parenting and collection structure.

Applied example:

PRD_Speaker_BODY, PRD_Speaker_KNOB and MAT_Plastic_White follow a shared convention and export consistently.

Review decisions:

- Publish a project convention sheet.
- Validate naming automatically where possible.
- Use origin and pivot rules by asset type.
- Record coordinate conversion.

3.3 Dimension and tolerance evidence

Visual models become accountable design artefacts when key dimensions, clearances and tolerances are measured and documented.

- **Nominal:** Target size from the brief.
- **Actual:** Measured Blender dimension.
- **Tolerance:** Permissible variation.
- **Clearance:** Required gap between parts.

Applied example:

A knob diameter is 32.0 mm ±0.5 mm and its body clearance is at least 1.0 mm after applied modifiers.

Review decisions:

- Measure critical-to-function features first.
- State whether modifiers affect measured output.
- Use appropriate precision.
- Flag assumptions not supported by source data.

3.4 Render engines and delivery intent

Choose Eevee for speed and interactive review, Cycles for physically richer final rendering, and Workbench for structural evidence.

- **Workbench:** Fast topology and form review.
- **Eevee:** Real-time look development and previews.
- **Cycles:** Path-traced lighting and material realism.
- **Viewport:** Immediate inspection, not final evidence by default.

Applied example:

A topology review uses Workbench wireframe; the design review uses Eevee; the final marketing still uses Cycles.

Review decisions:

- Match engine to decision purpose.
- Use Workbench overlays for mesh evidence.
- Control samples and denoising.
- Record engine and colour settings.

3.5 Camera and composition

A consistent camera turns renders into comparable evidence and communicates scale, function and hierarchy.

- **Focal length:** Controls perspective compression.
- **Framing:** Shows whole form with intentional margins.
- **Angle:** Reveals relevant features.
- **Consistency:** Locks comparison across variants.

Applied example:

A 70 mm three-quarter camera is duplicated across variants so differences come from design, not perspective.

Review decisions:

- Use orthographic views for dimensions.
- Use moderate focal lengths for product perspective.
- Lock camera after approval.
- Render required evidence views systematically.

3.6 Colour management

Colour management maps scene-linear values to the display; consistent settings are essential for fair review and predictable output.

- **Exposure:** Overall scene brightness.
- **View transform:** Maps dynamic range to display.
- **Look:** Optional contrast treatment.
- **Output:** Colour space and file format.

Applied example:

All approval renders use the same AgX-based view settings, exposure and studio lights.

Review decisions:

- Lock settings before variant comparison.
- Avoid editing material values to compensate for exposure.
- Use lossless output for evidence masters.
- Document display limitations.

3.7 Export formats

File formats preserve different subsets of geometry, materials, hierarchy, animation and units.

- **glTF/GLB:** Efficient real-time delivery with PBR materials.
- **FBX:** Broad DCC/engine interchange, but settings vary.
- **OBJ:** Simple geometry and UV interchange.
- **STL:** Triangle surface for printing; little scene metadata.

Applied example:

A web asset ships as GLB with packed PBR textures; a print test ships as millimetre-scale STL after manifold validation.

Review decisions:

- Choose by required data, not familiarity.
- Apply target axis and scale settings.
- Embed or package textures deliberately.
- Re-import to verify.

3.8 Game-ready optimization

Real-time assets balance silhouette, shading, textures and draw calls against a defined platform budget.

- **Geometry:** Triangle count and LODs.
- **Materials:** Minimise slots and draw calls.
- **Textures:** Resolution, compression and atlasing.
- **Collision:** Simple collision representation.

Applied example:

The hero asset uses one 2K texture set and three LODs; a simple convex collision mesh is delivered separately.

Review decisions:

- Triangulate predictably before final check.

- Bake high-detail normals onto low geometry.
- Use consistent texel density.
- Test inside the target engine.

3.9 3D-print preparation

Print preparation turns a visible surface into a manufacturable volume with scale, thickness, orientation and support considerations.

- **Volume:** Closed and manifold.
- **Thickness:** Meets process/material limits.
- **Overhang:** Compatible with orientation and supports.
- **Tolerance:** Allows fit after process variation.

Applied example:

A hollow product shell gains 2.0 mm wall thickness, drain holes and a flat print orientation before STL export.

Review decisions:

- Set units to millimetres deliberately.
- Check thin walls and disconnected shells.
- Orient to reduce support and weak layers.
- Use a slicer as a second validation environment.

3.10 Animation-ready organization

Animation depends on predictable pivots, transforms, topology, hierarchy and deformation—not merely a visually complete mesh.

- **Pivots:** Rotate around meaningful mechanical centres.
- **Hierarchy:** Parent-child motion relationships.
- **Topology:** Supports deformation where needed.
- **Naming:** Enables rigging and export automation.

Applied example:

A control knob remains a separate object with its origin on the rotational axis and a clear parent relationship.

Review decisions:

- Test one representative motion.
- Apply scale before rigging where appropriate.
- Keep moving parts separate.
- Avoid modifiers unsupported by the target.

3.11 Documentation package

A production-ready handoff includes the editable source, exports, previews, specifications, provenance and validation evidence.

- **Source:** Clean Blender master with organised data.
- **Exports:** Target-specific files and textures.
- **Preview:** Consistent renders or turntable.

- **Record:** Brief, changes, checks and known limitations.

Applied example:

The handoff contains SOURCE, EXPORT, TEXTURES, PREVIEW and EVIDENCE folders plus a readme and acceptance checklist.

Review decisions:

- Use a predictable folder structure.
- Include version and date.
- Separate final from working files.
- State open risks explicitly.

3.12 Automated checks

Small scripts can standardise repetitive checks such as naming, transforms, dimensions and object counts, while human review remains essential.

- **Rules:** Machine-checkable requirements.
- **Report:** Object-level findings with severity.
- **Repair:** Optional and narrowly scoped.
- **Review:** Human confirmation of geometry and intent.

Applied example:

A validation script reports unnamed objects, non-unit scale and missing UV maps; it does not automatically modify them.

Review decisions:

- Run checks without silent mutation.
- Show exactly what failed.
- Allow thresholds to be configured.
- Version the report with the asset.

3.13 Stakeholder design review

A review presentation should connect requirements, options, evidence, risks and recommendation in a decision-ready story.

- **Context:** Problem and intended use.
- **Options:** Comparable design variants.
- **Evidence:** Metrics and validation views.
- **Decision:** Recommendation, trade-offs and next actions.

Applied example:

The review recommends Variant B because it passes all functional criteria with a modest polygon increase and lower manufacturing risk.

Review decisions:

- Lead with the decision required.

- Show comparable views.
- Distinguish concept from verified evidence.
- Make unresolved risks visible.

3.14 Organisational alignment

An asset succeeds when it supports the organisation's audience, brand, production capacity, budget, risk appetite and reuse strategy.

- **Audience:** User needs and accessibility.
- **Brand:** Form language and visual consistency.
- **Operations:** Tools, skills and production constraints.
- **Lifecycle:** Reuse, maintenance and future variants.

Applied example:

A modular product model enables colour variants without duplicating geometry, reducing future campaign effort.

Review decisions:

- Evaluate business fit alongside technical quality.
- Prefer reusable modular structures.
- Capture ownership and maintenance.
- Reject novelty that increases risk without value.

3.15 Final acceptance and release

Release is a controlled decision based on complete evidence, resolved blockers and explicit acceptance of any remaining limitations.

- **Verify:** Run the full checklist on the release candidate.
- **Approve:** Record competent reviewer sign-off.
- **Package:** Freeze source, exports and evidence.
- **Archive:** Retain prior versions and provenance.

Applied example:

v08 is frozen after round-trip export, render, scale and topology checks; the handoff is signed and v07 archived.

Review decisions:

- Never approve a moving target.
- Re-run checks after the last change.
- Use checksums or immutable version labels.
- Separate answer keys and trainer-only material from learner publication.

Activity 9: Create a Neutral Studio Render and Visual QA Set

Goal: Use consistent cameras, lighting and colour settings to create comparable validation evidence.

Scenario: The design team needs evidence renders that reveal form and surface quality rather than hiding defects with dramatic lighting.

Output: A four-view neutral render set and a documented render configuration.

Step-by-step

1. Open the material-complete asset and save a render-working copy.
2. Create a neutral floor and light background with no distracting branding.
3. Add a large area key light, softer fill and restrained rim light.
4. Set a moderate product-visualisation focal length and frame the whole object with consistent margins.
5. Create front, side, top/orthographic and three-quarter camera views.
6. Lock the camera transforms and duplicate them for variant comparisons.
7. Choose Workbench for wireframe evidence, Eevee for quick review and Cycles for the final still as appropriate.
8. Set colour management, exposure, resolution, samples and output format.
9. Render a low-sample proof and inspect highlights for pinching, seams or normal errors.
10. Correct any blocking surface issue and rerun the relevant geometry check.
11. Render the approved four-view set using identical lighting and exposure.
12. Record the engine and key settings in the evidence presentation.

Acceptance checklist

- Four required views complete
- Camera and exposure consistent
- Neutral lighting reveals form
- Render settings recorded
- Blocking artefacts resolved

Evidence to submit

- Camera setup
- Lighting diagram
- Four-view render set
- Settings table
- Surface QA note

Activity folder: activities/09-…/

Activity 10: Prepare and Verify a Game/Visualization Export

Goal: Package a model for real-time delivery and verify it through round-trip import.

Scenario: A client needs a GLB asset for a web product viewer with predictable scale, materials and performance.

Output: A validated GLB export, round-trip Blender check and delivery record.

Step-by-step

1. Confirm the target viewer, forward/up axes, unit scale, triangle budget and supported material channels.
2. Duplicate the approved master into an EXPORT working file.
3. Remove or disable hidden source collections that must not ship.
4. Apply only the modifiers required for predictable export and preserve the editable master.
5. Verify object names, origins, transforms and material slot counts.
6. Pack or link textures according to the GLB delivery requirement.
7. Triangulate a duplicate predictably and record the final triangle count.
8. Export the selected objects to GLB with the agreed axis and material settings.
9. Create a new empty Blender file and import the exported GLB.
10. Compare bounding-box dimensions, object hierarchy, material appearance and normals with the source.
11. Open the file in the target viewer when available and record any differences.
12. Correct the source or export settings, then repeat the round-trip test.
13. Package the GLB, textures if separate, preview and readme.

Acceptance checklist

- Target requirements stated
- Only approved objects exported
- Scale and axes correct
- Materials survive round trip
- Triangle budget met

Evidence to submit

- Export settings
- Final statistics
- Source/import comparison
- Target-viewer screenshot
- Delivery package

Activity folder: activities/10-…/

Activity 11: Prepare and Verify a 3D-Printable STL

Goal: Convert a model into a watertight, correctly scaled print candidate with manufacturability evidence.

Scenario: The product concept must be delivered as a small physical prototype for ergonomic review.

Output: A millimetre-scale STL, printable-source Blender file and slicer-readiness checklist.

Step-by-step

1. Confirm intended print size, material/process, minimum wall thickness and required clearances.
2. Duplicate the approved source and remove decorative geometry not intended for printing.
3. Set scene units and verify the target dimensions in millimetres.
4. Join or boolean parts only where the physical prototype must be one connected volume.
5. Add or verify wall thickness and inspect narrow features against the process limit.
6. Use Non-Manifold selection and Face Orientation to verify a closed outward-facing surface.
7. Check for disconnected shells, internal faces and self-intersections.
8. Choose an orientation that reduces support and protects critical surfaces.
9. Apply required modifiers in the export copy while preserving the editable master.
10. Export STL using the documented unit setting.
11. Re-import the STL into an empty Blender file and verify dimensions and surface orientation.
12. Open the STL in a slicer when available and record warnings, support estimate and print time.
13. Package the STL, source file, screenshot and limitations note.

Acceptance checklist

- Print process and limits stated
- Closed manifold volume
- Wall thickness meets target
- Re-import scale correct
- Slicer warnings resolved or documented

Evidence to submit

- Dimension screenshot
- Manifold/Face Orientation checks
- Wall-thickness evidence
- Re-import comparison
- Slicer or readiness report

Activity folder: activities/11-…/

Activity 12: Capstone: Deliver an AI-Assisted 3D Asset Review Package

Goal: Integrate brief, AI assistance, Blender refinement, validation, export and stakeholder recommendation.

Scenario: You are accountable for a complete product-asset handoff that will be reviewed by design, production and business stakeholders.

Output: A release-candidate Blender asset, target export, validation dashboard and decision-ready presentation.

Step-by-step

1. Select one approved design brief and define the final target use.
2. Record the AI contribution: concept reference, prompt, script or variation support.
3. Create or refine the model in Blender using named objects and organised collections.
4. Freeze a baseline before topology or material repair.
5. Run the validation framework across dimensions, transforms, topology, normals, UVs, materials and naming.
6. Prioritise defects and repair blocking items using versioned files.
7. Optimise the asset for the target polygon, print or render requirement.
8. Create a neutral evidence render set with fixed cameras and settings.
9. Export to the target format and complete a round-trip verification.
10. Populate the final quality dashboard with thresholds, results and evidence links.
11. Prepare the sample presentation: context, approach, variants, validation, result, risk and recommendation.
12. Run the activity checklist and have a peer review one evidence item against the brief.
13. Resolve or explicitly accept remaining limitations and record the owner.
14. Freeze the release candidate, copy final exports into the delivery package and archive working versions.

Acceptance checklist

- All K/A areas evidenced
- Baseline and versions traceable
- Blocking checks passed
- Round-trip export verified
- Recommendation and limitations clear

Evidence to submit

- Brief and provenance
- Version history
- Validation dashboard
- Final render set
- Round-trip result
- Stakeholder presentation
- Release package

Activity folder: activities/12-…/

Quick Reference

Blender action	Purpose
Object Mode / Edit Mode	Switch between whole-object and component editing.
N panel > Item	Inspect transforms and dimensions.
Viewport Overlays > Statistics	Read vertices, faces and triangles.
Face Orientation	Find inward-facing polygons.
Select > Select All by Trait > Non-Manifold	Locate open or ambiguous surface elements.
Ctrl+A > Scale	Apply scale only after dimension verification.
UV Editing workspace	Mark seams, unwrap and inspect distortion.
Shader Editor	Build and inspect PBR material nodes.
File > Export	Create target-specific GLB, FBX, OBJ or STL deliverables.
Scripting workspace	Review and run Blender Python scripts.

Assessment and Support

Written Assessment (SAQ): 6 open-ended knowledge questions, 1 hour, open book.

Practical Performance (PP): 3 integrated Blender tasks, 2 hours, open book.

3 hours on Day 4, from 3:00 PM to 6:00 PM.

- Complete TRAQOM / SSG assessment attendance.
- Sit the Written Assessment, then the Practical Performance.
- Upload completed answers to https://lms-tms.tertiaryinfotech.com/
- Sign the Assessment Summary Record.

Support: enquiry@tertiaryinfotech.com · +65 6100 0613